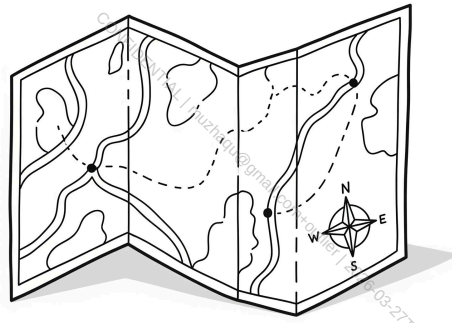




Menu Territory

Attempter Guidelines

Change Log

 Date	 Updates	 Notes/Images/Links
Mar 25, 2026	→ Updated guidelines not to include any negative criteria in the rubrics section.	
Mar 24, 2026	→ For every task, use the repo link with base and target commits to get the unified diff which needs to be pasted in the prompt section as the reference text. → Updated  Golden Review Response and Rubrics Alignment that the "golden response must pass all the rubrics" which will be evaluated by the model.	
Mar 23, 2026	→ For the truncated or invalid diff in the provided unified diff, complete the code using the PR link.	
Mar 17, 2026	→ Updated Step 4 : Generate Rubrics based on the golden response	
Mar 16, 2026	→ Every task must include at least one blocker issue and non-blocking suggestions, along with the corresponding rubric.	
Mar 13, 2026	→ All the rubrics should be atomic . → Avoid Overlapping or Redundant Criteria	

 Date	 Updates	 Notes/Images/Links
Mar 12, 2026	→ There should be AT LEAST one rubric for non-blocking issues, and it should follow a format like "The response suggests non-blocking issues such as XXX."	
Mar 10, 2026	→ The new batch will no longer include the previous "review + fix" type of tasks → For the " Where " part of each blocker, providing a specific line range is no longer required.	
Mar 3, 2026	→ Edited Weighting Rules section → Added Golden Review and Rubrics Alignment section	
Feb 27, 2026	→ Updated golden review and rubrics guidelines → Updated Common Errors to Avoid	
Feb 24, 2026	→ Updated the rubric template for potential fixes	
Feb 18, 2026	→ Repo is provided. Changes are made as the task artifacts are provided in the task. → Good Prompt Examples section added.	
Feb 2, 2026	→ Initial version.	

Contents

Introduction

Guidelines

[STEP 1: Read Task Specs & Constraints](#)

[STEP 2: Draft the Prompt](#)

[STEP 3: Generate Golden Response based on PR](#)

[STEP 4: Generate Rubrics based on the golden response](#)

[STEP 5: Evaluate the Model Evaluation based on Rubrics](#)

Appendix

Introduction

Welcome to Menu Territory Code Review! The following document establishes the general expectations for the project. These guidelines apply to all task types.

This project has the following goals:

- Provide a complex prompt based on the provided task artifacts and create rubrics that challenge cutting-edge AI models.
- Provide a Golden Response based on the unified diff in the prompt section.
- Evaluate a model's ability to identify all issues in the unified diff, and provide helpful feedback (e.g., propose fixes) through the creation of Rubrics.
- Verify the created prompt, and rubrics stumped at least 1 model.

Guidelines



Important Tip

Use whatever workflow you prefer to browse PRs and view commit hashes/diffs. The output must be reproducible from public repo data.

Step 1: Read Task Specs & Constraints

1) Repo and other task artifacts provided in the task

Review all the task artifacts (repo, issue title, diff, PR link, etc.) provided in the tasks:

- ✓ All required information will be provided to you.

2) Save the diff in the prompt

- For every task, use the repository link with base and target commit IDs using the format below to get the unified diff.

Format: **repo link/compare/<base commit>...<target commit>.diff**

- This opens the raw diff directly. Copy/paste the entire diff from that link to your **prompt section as reference text**.

Example:

`https://github.com/[repo_placeholder]/compare/<base_commit>...<target_commit>.diff`

Note*: You do not have to use or check the pre-provided diff at all.

STEP 2: Draft the Prompt

The prompt should assign a persona of a senior software engineer (or equivalent) tasked with performing a code review. A review of the unified diff must be clearly requested.

The (Diff) reference text of the prompt must have issues, so that there is something to point out in the code review. **There must be at least one serious blocking issue in the given diff.** If there are no serious issues in the diff, please flag the task and skip the task. Here is a list of the issues that are allowed:

Serious Blocking Issues	Non-serious Issues
→ new bugs/correctness → security risk → functionality failures → missing core tests → core edge cases → performance regression → API misuse → Etc	→ poor formatting → dependency bumps → subpar documentation → Etc

To craft your prompt, make sure to give the following structure:

Role + goal (2–3 sentences)

"You are a senior software engineer reviewing a unified diff to verify it solves the problem statement, is production-ready, and introduces no bugs."

Problem statement (3–8 lines)

Explain what's broken / what's expected. Keep it concrete.
Example:

None

The current data import process (specifically for Slack and generic imports) relies on passing around unstructured dictionaries to track upload records. This lack of structure makes the code brittle, difficult to type-check, and prone to key errors. The diff refactors the uploads processing code paths to use a structured dataclass named UploadRecordData instead of raw dictionaries. It enforces better typing and cleaner data passing

To craft your prompt, make sure to give the following structure:

between the import parsers and the processing utilities.

Diff saved

The diff saved as reference text in the prompt should contain the entire diff from the repository link using the format below.

Format: `repo link/compare/<base commit>...<target commit>.diff`

Output requirements

This section specifies the mandatory structure required for a "Golden Response." The model must strictly adhere to these formatting guidelines to ensure consistency and technical depth.

Example:

None

Provide the detailed diff review with the following structure:

Summary

Blockers (for each blocker, include what is wrong, why it matters, how to fix it, and where it exists in the diff)

Non-blocking suggestions

Tests/Validation

STEP 3: Generate Golden Response based on unified diff

Write the Golden Review as the ideal senior-engineer unified diff review comment. It must be structured, specific, and actionable, and it must match the task's required output.

The Golden Review should encapsulate what an ideal response should look like — **it shouldn't fail any of the Rubric's criteria**, and **should be free of any objective inaccuracies**, and its **code changes should not introduce errors**.

Required Structure	
1. Summary (2-4 sentences)	→ Does the patch solve the problem statement? → What are the top 1-2 issues that matter most?
2. Blockers (Serious blocking issues)	→ Bullet list → For each blocker, include: • What's wrong • Why it matters (bug, regression risk, correctness, UX, etc.) • Exact fix (conceptual solutions on what to change) • Where (file path provided in the diff) → Each task must include at least one blocker
3. Non-blocking suggestions	→ Optional improvements (readability, small cleanup, formatting, documentation, dependency bumps, etc). → Keep it short and avoid scope creep.
4. Tests / Validation	→ List the test cases or validation cases for the specific edge cases that must be tested. → If the changes affect behavior, require a regression test or some other tests as needed.

STEP 4: Generate Rubrics based on the golden response

Create a **minimum of 15** rubric items that cover:

- 1. Core correctness** (must-catch bug/issue as blocker)
The response identifies the actual problem and why it breaks expected behavior.
- 2. Required fix for blockers**
The response proposes the correct change(s) that fully solve the stated problem.
E.g., "The response suggests a fix for <blocker_x>, for example <suggested_fix>"
- 3. Optional improvements**
The response proposes Optional improvements to make the code better.

E.g., "The response suggests non-blocking issues such as XXX." or
"The response includes a non-blocking suggestion that XXX."
- 4. Regression/edge-case coverage**
The response asks for or provides tests that cover the critical edge cases.
E.g., "The response recommends performing a test that ..."
- 5. Clarity / actionable review**
The response is structured, following the format requested from the prompt.

E.g., "The response follows a structure with Summary, Blockers, Non-blocking Suggestions, and Tests/Validations sections."

6. Hygiene checks when applicable

The response flags obvious issues like debug prints, conflicting CLI flags, missing prompts/required args, etc.

There should be no negative criteria in the rubrics section. Do not use any criteria like shown in the examples below:

- ✗ The response does not suggest unrelated refactors or changes outside the scope of the Smooth Linear Advance feature and its integration with existing Marlin subsystems.
- ✗ The response does not flag issues in files not modified by the diff or demand changes outside the two identified blockers.
- ✗ The response does not suggest unrelated refactorings or changes outside the scope of this PR.
- ✗ The response does not flag the RegionMetadataLoader move from opener.rs to utils.rs as a bug or blocker, since it is a straightforward relocation with no logic changes.
- ✗ The response does not suggest changes unrelated to the exclusion rules merging, popup/options unification, or template loading introduced by this PR, such as modifying keyboard navigation, link hints, or find mode.

The rubrics for specific or explicit requests in the prompt should avoid, at all costs, being subjective, too vague, overly broad, or unmeasurable (e.g., "the response should have good formatting" or "code must be optimal"), and they should also cover every single serious issue in the diff.

These rubrics should never check for something that contradicts what's explicitly required in the prompt, contain a factual error or misleading point, or check something that is not either an explicit or implicit requirement from the prompt, and including it in the Golden Review does not improve anything.

Please ensure there are no missing criteria: all blocker issues/potential fixes, as well as non-blocking suggestions and tests/validation, must have corresponding criteria.

When writing rubrics for potential fixes, focus on describing necessary modifications and proposing conceptual solutions similar to a human review rather than strictly requiring exact corrected code; please ensure it is evaluated using a generalized rubric item "The response suggests a fix for <blocker_x>, for example <suggested_fix>" to maintain flexibility and accommodate multiple valid implementation methods.

All the rubrics should be self-contained. Ask yourself: "Using **only the rubrics, and the Model Response**, am I able to tell that the Model Response is correct?" If yes, you can include that rubric. But if not, and you think that you should have access to **external info (like a diff file or the prompt)** to determine if the response is correct or not, then that rubric is not self-contained, and it should not be included.

All the rubrics should be atomic. Each rubric should evaluate one thing only — no bundling of multiple behaviors. Ask yourself if the criterion is evaluating more than one idea. A rubric should not contain two complete sentences under any circumstances.

Finally, we should **avoid Overlapping or Redundant Criteria**. A criterion that is either:

- Completely redundant because other criteria already cover it, or
- Partially overlapping where multiple criteria check for the same thing.

Redundant Scenario:

Criteria 1: Response does a, b, c

Criteria 2: Response does a, b

Overlap Scenario:

Criteria 1: Response does a, b, c

Criteria 2: Response does b, c, d

Rubric Types

Instruction Following	Ensures the response adheres to explicit directions in the prompt (formats, constraints, language, libraries, required elements).
Bug Discovery	Verifies the model identifies and suggests solutions for bugs or issues present in the PR.
Logic Reasoning	Verifies the model identifies and suggests solutions for any reasoning flaws.
Style and Tone	Checks for any formatting or tone requirements that would make the response better looking.

Weighting Rules

Each rubric criterion must have one of the following weights:

5 — Mandatory	- Core requirement for adequately answering the prompt. - Response would be unacceptable without it. - Explicit requirement from the prompt. - Recognition or identification of serious blocking issues.
----------------------	--

Weighting Rules

Each rubric criterion must have one of the following weights:

	- Possible solution or fix for new bugs/correctness issues, functionality failures, API misuse, security risks, missing core tests, performance regression, core edge cases, etc.
3 — Important	- Makes the response substantially better. - Implicit requirement from the prompt. - Response would be acceptable without it if all other criteria are met. - Recognition of non-blocking issues and suggestions, tests/validations.
1 — Nice to have (Optional)	- Nice-to-have and necessary for a perfect response. - Response can still be strong enough without it. - Could include best code practices, libraries, language versions, formatting, style/tone etc which are not asked in the prompt explicitly.

Rule of Thumb for Rubric Types and Weighting

1. Prompt-Specified Section Requirements → Mandatory (Instruction Following)

The section requirement explicitly stated in the prompt (e.g., "The response follows a structure with Summary, Blockers, Non-blocking Suggestions, and Tests/Validations sections") should be treated as **Mandatory** and categorized under **Instruction Following**.

2. Blocker Issues → Mandatory (Bug Discovery)

All bugs identified as **blockers**, along with their corresponding fixes, must be included as **Mandatory** criteria under **Bug Discovery**.

3. Non-Blocking Suggestions → Importance-Based Classification

The weighting and rubric type depend on the nature and impact of the suggestion:

- **Minor format/style issues** (e.g., docstrings, grammar in **.md** files):

- Type: **Style and Tone**

- Weight: **Important**

- **All other meaningful suggestions** (e.g., design, robustness, maintainability):

- **Weight: Important**
- **Type: Choose based on content:**
 - **Bug Discovery** (if it prevents potential issues)
 - **Logic Reasoning** (if it improves design)

4. Tests / Validation → Important or Mandatory based on whether they are mentioned in the blockers (Instruction Following)

- **Mandatory** (core tests missing, listed in Section 2: Blockers), or
- **Important** (For other tasks listed in Section 4: Tests/Validation)
 - **Common pitfall:** The same test should not appear across multiple sections, and multiple rubric criteria should not evaluate the same test (overlapping issue)

Write criteria that encompass all requirements needed to fulfill this prompt 1/15 completed

1 The response includes an...

Category *

If unsure, reference the category definitions.

Options [View details](#)

- ☐ Instruction Following
- ☐ Bug Discovery
- ☐ Logic Reasoning
- ☐ Style and Tone

How Does This Criterion Affect The Response? *

Options [View details](#)

- ☒ Mandatory
- ☐ Important
- ☐ Nice to Have

Weight (Mandatory) *

This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.

5

STEP 5: Evaluate the Model Evaluation based on Rubrics

Does at least one of the models score below 70% AND the average of the three < 70%?

- **If yes**, you can proceed with labeling and finishing the task
- **If not**, you need to tweak the rubrics and/or prompt to increase the complexity

Assistant

Rubric Evaluation Viewer

	2nd place 75%	3rd place 25%	★ 1st place
Response A Finger painting is a great activity for kids ages 3–5. It encourages...	Response B Try building a blanket fort together! Kids can help arrange...	Response C When it's raining, kids can set up...	
Criterion 1 The response suggests an indoor activity appropriate for 3–5 year-olds (e.g. building with blocks,...	No	Yes	Yes
Criterion 2 The response mentions how the activity helps with development (e.g. fine motor skills, creativity,...	Yes	No	Yes
Criterion 3 The response acknowledges that the activity is suitable for a rainy day (e.g. "indoors," "rainy day,"...	Partially	Yes	Yes

Assistant

Do at least one of the models score below 70% AND the average of the three < 70%?

☐ Read only

Do at least two of the models score below 65% AND the average of the three < 65%? *

(You can select 'No' and click 'Save' to see the average score)

☐ Yes

☐ No

Choose the difficulty of the task

☐ Hard

☐ Medium

☐ Easy

Golden Review Response and Rubrics Alignment

 Remember, the golden review you have written should **pass all the rubric criteria** that you have created, including Mandatory criteria, Important criteria, and Nice to Have

criteria. Your task will be considered a failure and rejected if the golden review fails even one of the rubric's [Mandatory or Important] criteria, and it will receive a maximum review score of only 3 if it fails even one [**Nice to Have**] criterion.

Please give a special focus to align your golden review, prompt, and rubrics based on the unified diff provided in the task.

 **Assistant**

If golden response does not pass all the rubrics then please fix the golden response.

 Read only

Does Golden Response pass all the rubrics? *

Does Golden Response pass all the rubrics? If no then please fix the golden response.

☐ Yes

☒ No

Helpful Tips:

- Always make sure the golden review response fulfills the prompt request completely.
- Always make sure the golden review response and the prompt is aligned with the unified diff.
- While creating rubrics, if you add criteria that are not covered or fulfilled by the golden review, go back to your golden review and make sure that it passes the new criteria you have added.

✓ Good Prompt Examples

Some examples of good prompts for this project are provided below. Use them as references, but **do not** try to use them directly in your task.

Prompt #1:

None

You are a senior software engineer reviewing a unified diff in the Healthchecks repository. Healthchecks is a production cron job monitoring service built with Python 3.12, Django, and a relational database (PostgreSQL, MySQL, or MariaDB).

Your role is to verify that the changes correctly solve the stated problem, are production-ready, and do not introduce regressions or subtle bugs.

Healthchecks relies on incoming "ping" requests to track the execution of scheduled tasks. Currently, the system cannot identify individual executions of overlapping or retried jobs, which leads to incorrect duration calculations and duplicated ping records. When clients retry failed requests, multiple identical pings can be logged, breaking idempotence and inflating execution history. Users running overlapping cron jobs cannot reliably track the start and end times of each run, and the check state can become inconsistent when one run finishes while another is still active.

This diff introduces support for a user-specified "run id" parameter to enable accurate tracking of overlapping runs and to lay groundwork for making ping requests idempotent.

A full unified diff is provided. Review the diff as the sole source of code changes.

Output format:

- 1) Summary
- 2) Blockers (for each, state what's wrong, why it matters, how to fix it, where it exists)
- 3) Non-blocking suggestions
- 4) Tests/validation

Prompt #2:

None

You are a senior software engineer reviewing a unified diff in the Zulip repository. Zulip is an open-source team chat application built with Python 3 and Django. Your goal is to verify that the refactoring of the data import system improves code quality and type safety without introducing regressions, side effects, or bad practices.

The current data import process (specifically for Slack and generic imports) relies on passing around unstructured dictionaries to track upload records. This lack of structure makes the code brittle, difficult to type-check, and prone to key errors. This diff refactors the uploads processing code paths to use a structured dataclass named `UploadRecordData` instead of raw dictionaries. The goal is to enforce better typing and cleaner data passing between the import parsers and the processing utilities.

A full unified diff has been provided as reference text. Return a detailed review of the unified diff as the sole source of code changes. Do not assume any changes beyond what is shown.

Output format:

- 1) Summary
- 2) Blockers (for each, state what's wrong, why it matters, how to fix it and where it exists)
- 3) Non-blocking suggestions
- 4) Tests/validation